

DeliveryFlow.API

Documentação Técnica e Arquitetural

Projeto desenvolvido utilizando .NET 8 seguindo conceitos de Clean Architecture, SOLID, separação de responsabilidades e boas práticas de desenvolvimento.

Tecnologias Utilizadas

Tecnologia	Descrição
ASP.NET Core	Desenvolvimento da API REST
Entity Framework Core	Persistência de dados
JWT	Autenticação e autorização
Serilog	Logs estruturados
xUnit	Testes unitários
FluentValidation	Validação de entradas

Objetivo da Solução

O DeliveryFlow.API foi desenvolvido para atender às necessidades da empresa fictícia TechsysLog, especializada em logística e gerenciamento de entregas. A solução permite gerenciamento de usuários, pedidos e registros de entrega através de uma API REST, priorizando organização, escalabilidade, segurança e manutenção.

Arquitetura e Organização da Solução

A aplicação foi estruturada seguindo conceitos inspirados em Clean Architecture, separando responsabilidades em diferentes camadas para reduzir acoplamento e facilitar evolução futura. A organização também segue princípios do SOLID, principalmente: - Single Responsibility Principle (SRP) - Open/Closed Principle (OCP) - Dependency Inversion Principle (DIP) A separação das camadas foi realizada para garantir maior reutilização, desacoplamento entre componentes e facilidade de testes unitários.

Camada API

Responsável pela entrada da aplicação e exposição dos endpoints REST. Principais componentes: - Controllers - JWT - Swagger - Middlewares - Dependency Injection - Program.cs - appsettings.json
Essa camada possui responsabilidade exclusiva de lidar com requisições HTTP e comunicação externa da aplicação.

Camada Application

Contém toda a lógica de negócio da aplicação. Estrutura: - Services - Validators - DTOs - Interfaces - Contracts - DomainErrors A camada Application foi separada da Infrastructure para evitar dependência direta de tecnologias externas como Entity Framework ou banco de dados.

Camada Domain

Representa o núcleo da aplicação. Contém: - Entities - Objetos compartilhados - Regras centrais de domínio O Domain não possui dependência de frameworks externos, garantindo isolamento das regras de negócio principais.

Camada Infrastructure

Responsável pela persistência de dados e integrações externas. Componentes: - Entity Framework Core - Repositories - Unit of Work - Migrations - DataContext - Fluent API Essa separação permite substituir tecnologias externas sem impactar diretamente as regras de negócio da aplicação.

Camada Shared

Responsável por centralizar componentes reutilizáveis entre os projetos. Contém: - Logs - Helpers - Responses - Identity customizado - Utilitários compartilhados

Autenticação e Segurança

A aplicação utiliza JWT (JSON Web Token) para autenticação. Regras implementadas: - Usuário autenticado obrigatório para operações de pedidos. - CRUD de pedidos protegido. - Usuários possuem permissões restritas. - Controle de acesso implementado via autenticação JWT.

Observabilidade

A solução possui observabilidade através do Serilog e middleware global de exceções. Benefícios: - Logs estruturados - Centralização de erros - Melhor rastreabilidade - Facilidade de troubleshooting

Testes Unitários

Foram implementados testes unitários utilizando xUnit. Classes testadas: - UserService - OrderService - UserRequestValidator - OrderRequestValidator Estratégia: - 1 cenário de sucesso - 1 cenário de falha para cada método testado.

Conclusão

O projeto foi desenvolvido buscando seguir boas práticas arquiteturais e padrões de desenvolvimento, garantindo organização, escalabilidade, manutenção facilitada e separação clara de responsabilidades. A estrutura criada permite evolução futura da aplicação de forma desacoplada e sustentável.
